

PARCELYT

(COURIER & PARCEL MANAGEMENT SYSTEM)

BY

NAME	REGISTRATION NO
M.RISHITHA	AP24110011630
A.LAYA CHANDANA	AP24110011636
CH.PRAGNYA SREE	AP24110011642
K.SOMASAISH	AP24110011658
V.KUNDANA	AP24110011661

1. INTRODUCTION :

In today's rapidly developing digital world, there is a high demand for efficient and reliable courier services. The conventional system involves a lot of manual work which is prone to mistakes and causes delay in deliveries and poor visibility. For this reason, the web-based Courier and Parcel Management System called Parcelyt has been developed.

The Parcelyt software provides an easy-to-use interface where users can sign up, make a booking for their parcels, and get real-time updates using a unique tracking ID number. In addition, users can check the delivery history of their parcels, get alerts when the parcel is delivered, and contact customer support whenever necessary.

For the administrators, the system features a wide range of options to manage their courier business effectively. They are able to monitor all parcels, change the status of the delivery from "In Transit" to "Delivered," collect and interpret data, and reply to all messages sent by the users.

The application is built utilizing the MERN stack, which consists of MongoDB, Express.js, React.js, and Node.js, providing an effective structure. The frontend part helps to achieve a highly interactive interface whereas the backend takes care of business logic and secure data processing. MongoDB is employed to store all user data and parcel data effectively.

Security and reliability are important characteristics of Parcelyt. Parcelyt employs a number of security measures including the implementation of JWT authentication, access control based on user roles, as well as data encryption. Parcelyt can be referred to as a comprehensive tool for managing couriers' work online.

2. OBJECTIVES :

First, the aim of developing Parcelyt Courier and Parcel Management System is to create a system that allows convenient and reliable management of courier services in a digital environment. Thus, the system seeks to solve limitations associated with manual approaches in booking parcels, tracing the status of the same, and the management of parcel delivery operations.

In addition to providing efficient and effective services in delivering parcels, another important objective of the system is real-time tracking of all parcels. This can be achieved through unique parcel tracking codes assigned by the system and delivery statuses recorded by the system throughout the process.

Another important aspect of the management system is to ease administration of processes. Through the system, one can easily track data concerning parcel details and delivery status of parcels and manage the efficiency of the process.

Security is another key concern of the system. As such, there is the implementation of various security measures that will ensure secure access and control of the system by only authorized persons. In addition, efficient handling of large data sets and the creation of consistent data sets are also considered important objectives of the system.

Project Specific Objectives:

- Development of a web application for booking and management of couriers' services.
- Implementation of real-time tracking service for packages with the use of tracking ID.
- Minimizing of mistakes through the automation of processes.
- Secure authentication for users and protection of their data.
- Communication channel between customers and admin personnel.
- Decision-making tool through statistics and analytics.
- Development of scalable systems with the use of up-to-date technologies (MERN stack).

3. SCENARIO BASE CASESTUDY :

Meet Rahul, a professional working individual who often requires sending parcels to various cities for personal and professional reasons. Traditional courier networks present many problems for Rahul such as long wait times, no tracking facility, and absence of information regarding delivery. Rahul resolves all these problems by adopting the Parcelyt Courier Management System, a contemporary web-based platform offering easy courier services.

User Account Creation and Authentication : To use Parcelyt, Rahul first creates a user account on the system using his email and password. His information is safely

stored in the system using encryption technology, allowing him to sign in at any time. Thanks to JWT authentication, Rahul's sessions are highly secured against any unauthorized access.

Booking the Parcel :Rahul then proceeds to book his parcel from the login interface. After entering the relevant details such as sender and recipient details, type of parcel, weight, and other necessary parameters, Rahul receives a tracking ID that helps him monitor the parcel at any point in time.

Tracking Real Time and Getting Notifications :While the parcel undergoes different phases such as "Booked," "Picked Up," "In Transit," and "Out for Delivery," Rahul can track its progress through the system's tracking mechanism in real time. Furthermore, Rahul will get notifications as soon as there is an update on the status of the parcel.

Monitoring and Managing by Admin :On the other hand, the system administrator can monitor the activities carried out for each parcel from the central dashboard of the system. The admin can update parcel statuses, as well as manage user data and deliver several parcels at once. Should Rahul make an inquiry, the admin would answer through the internal messaging platform provided.

Providing User Support and Feedback :If Rahul runs into any challenge, he can easily make use of the support feature in order to communicate his problems. In return, the admin would read and reply to his message. Upon successful delivery, Rahul may give feedback.

Outcome of the Project :Parcelyt has enabled Rahul to have a very convenient and smooth couriership process since he no longer needs to be afraid of delay and lack of information about his package.

4. SYSTEM REQUIREMENTS :

For efficient development, deployment, and utilization of the Parcelyt Courier and Parcel Management System, some preconditions need to be fulfilled. The requirements are grouped into software and hardware prerequisites, which facilitate the effective working of the application.

a. Software Requirements :

These are the necessary tools and technologies that are needed to develop and deploy the Parcelyt application.

- **Operating System:** Windows 10/11, macOS, or Linux – Facilitates development and testing on different platforms.
- **Node.js (version 16 and above):** Used for development of the back end and execution of server operations.

- **npm (version 8 and above):** Package manager used for installation of dependencies on front end and back end.
- **React.js:** Front-end technology used to build UI for the user dashboard and admin dashboard.
- **Express.js:** Back-end technology used for creating and managing APIs.
- **MongoDB:** NoSQL database used to store user account details, parcels, message data, and other information.
- **JWT (JSON Web Tokens):** Used for authentication and session management.
- **Browser:** Google Chrome/Mozilla Firefox – For executing the application and its testing.
- **Postman:** Used for testing the APIs developed in the back end.
- **Visual Studio Code:** IDE used to code and manage the project.

b. Hardware Requirements :

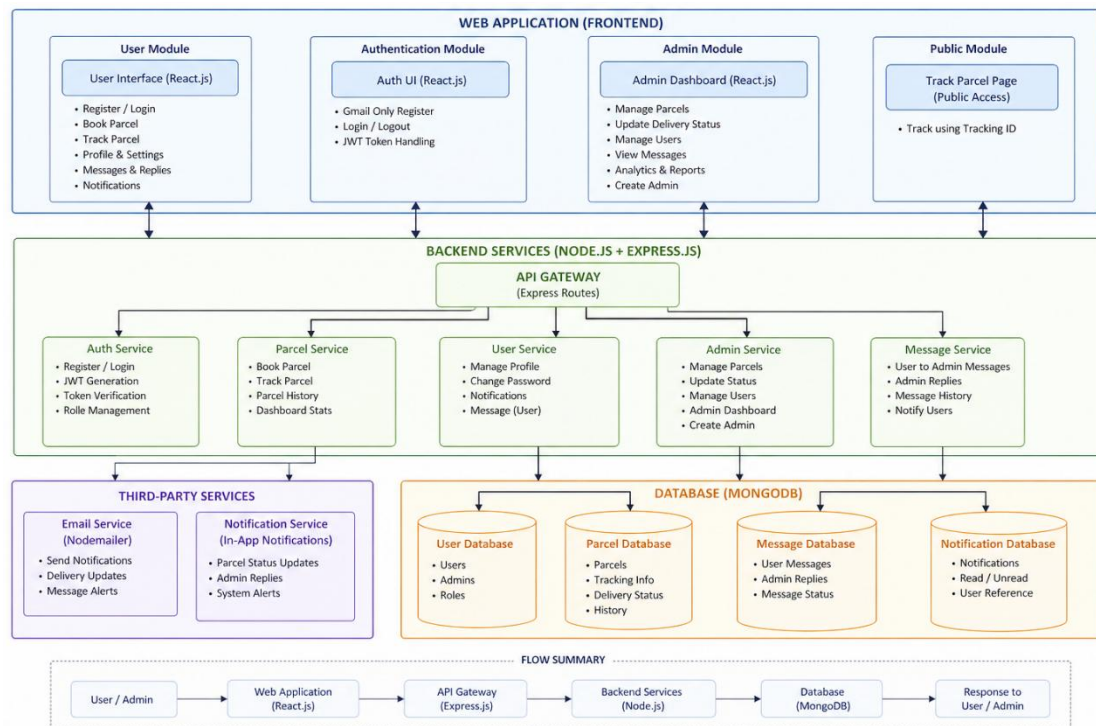
The following hardware specifications are necessary to support the Parcelyt application effectively.

- **Processor:** Intel Core i5 (8th generation or later) / AMD Ryzen 5 or better — To ensure that frontend and backend operations execute seamlessly.
- **RAM:** Minimum 8 GB (16 GB recommended) — Must be present for the simultaneous operation of the server, database, and browser.
- **Storage:** Minimum 1 GB available storage — Needs to be available for database and project installation purposes.
- **Display:** 1366 x 768 or greater resolution — Suitable display resolution to view the application interface.
- **Internet:** Necessary for installing software packages, communicating through APIs, and real-time parcel tracking features.

c. Additional Requirements :

- **Authentication Module:** JWT-based login for secure user authentication and access control for users/admins.
- **API Testing:** Necessary to test parcel booking, tracking, and admin functionalities.
- **Database Connection:** Requires stable connectivity between MongoDB and the application to store parcel information.
- **Browser Compatibility:** The application should work seamlessly on all modern browsers.

5. SYSTEM ARCHITECTURE :



In this architecture, the flow starts from the User/Admin, who accesses the Parcelyt system via the Web Application (Frontend) created on the basis of React.js. The frontend involves several components including the User Module, the Authentication Module, the Admin Module, and the Public Tracking Module. Users have the ability to register, log in, book parcels, track them, and send requests for assistance while administrators can manage parcels and oversee system activities.

From the frontend, requests are forwarded to the Backend Services, which are implemented using Node.js and Express.js. They are passed through the API Gateway to be delivered to the proper services, including the Authentication Service, the Parcel Service, the User Service, the Admin Service, and the Message Service. Every service handles certain tasks, namely, user login, booking of parcels, user management, updating the status of deliveries, and interaction between users and administrators.

The backend services communicate with the MongoDB Database, which serves as the storage place for all information. The database includes several collections, namely those containing users, parcels, messages, and notifications.

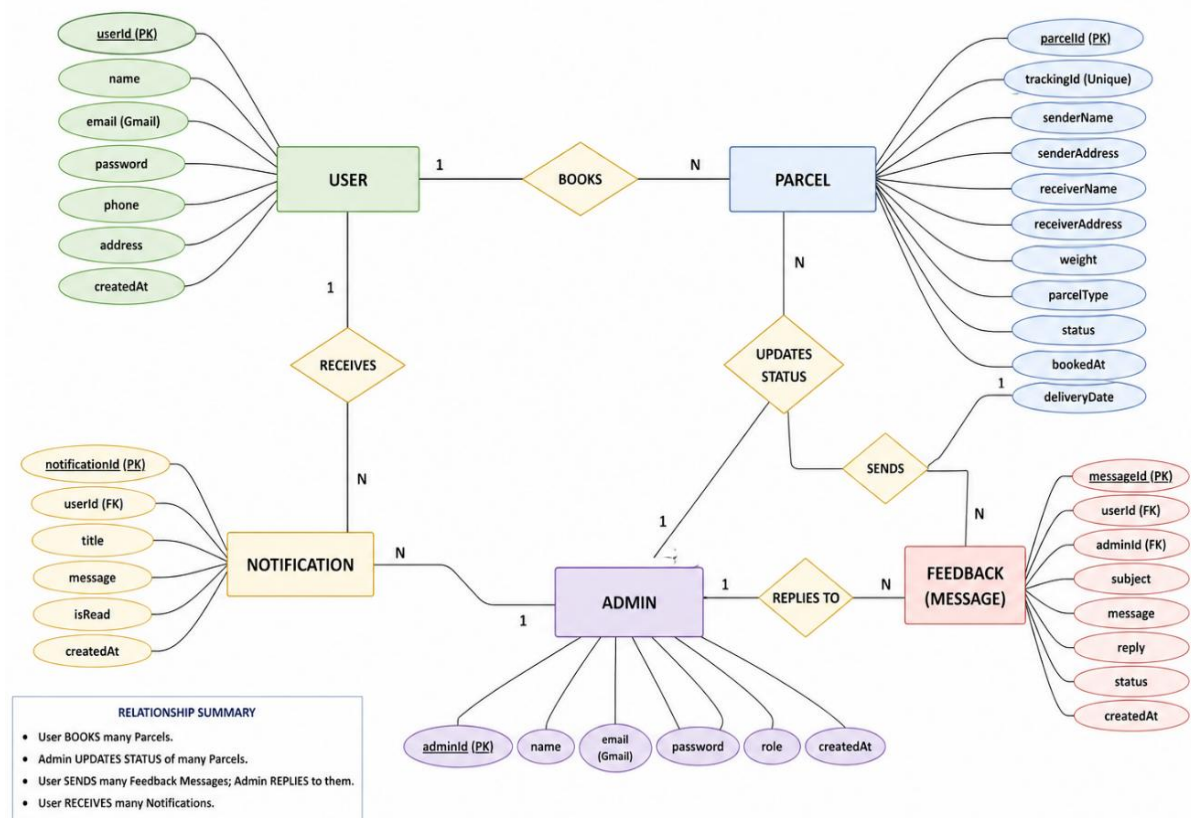
In addition to this, the system also integrates with **Third-Party Services** such as Email Service and Notification Service. These services are used to send alerts, delivery updates, and admin replies to users, improving communication and user experience.

Once the backend processes the request, it sends the response back through the API Gateway to the frontend. The frontend then displays the updated information to the user or admin, such as parcel status, notifications, or dashboard data.

Overall, this architecture ensures:

- Proper separation of frontend, backend, and database
- Secure communication using authentication mechanisms
- Efficient handling of user requests
- Scalability and flexibility using the MERN stack

6. ER DIAGRAM :



The ER Diagram for Parcelyt Courier and Parcel Management System illustrates different entities and their relationship with one another. It is a visual representation of how data can be stored in the database.

1. Entities:

- **User:** This entity symbolizes the users/customers of the system. The entities include a user ID, name, email, password, phone, address, and created date.
- **Parcel:** This entity symbolizes the parcels booked on the platform. The entities include a parcel ID, tracking ID, sender/receiver information, weight, parcel type, status, booking date, and delivery date.
- **Admin:** This entity symbolizes the administrator who controls the system. The entities include an admin ID, name, email, password, role, and created date.

- **Feedback (Message):** This entity symbolizes feedbacks/messages from users to the admins. The entities include message ID, user ID, admin ID, subject, message body, reply, status, and created date.
- **Notification:** This entity symbolizes alerts issued to the users. The entities include notification ID, user ID, title, message, read status, and created date.

2. Relationship:

- **User - Parcel (Booked):** One user can book many parcels. But only one parcel belongs to one particular user (One-to-Many Relationship).
- **Admin - Parcel (Updated Status):** One admin is managing the parcels by updating their statuses.
- **User – Feedback (Sent):** A user sends many messages to the admin.
- **Admin – Feedback (Replied To):** Admin replies to messages of users.
- **User – Notification (Received):** User gets many notifications about parcel update and response from admin.

3. Attributes:

Attributes define the properties of an entity. Such as:

- User → Name, Email, Password
- Parcel → Tracking ID, Status, Delivery Date
- Feedback → Message, Reply, Status

4. Primary Keys:

Every table has primary keys for unique identification of records in the tables.

- User → UserId
- Parcel → ParcelId
- Admin → AdminId
- Feedback → MessageId
- Notification → NotificationId

5. Foreign Keys:

- Foreign keys are used to connect two different tables.
- Parcel has foreign key userId which links the parcel with the user who booked it.
- Feedback table has foreign keys such as UserId & AdminId.
- Notification table has foreign key UserId.

SUMMARY:

It can be seen that there are several relationships among users, parcels, admin, message and notifications in the system Parcelyt with the help of this ER Diagram.

7. KEY FEATURES :

Parcelyt Courier and Parcel Management System offers several features aimed at improving the user experience and streamlining administrative work.

User Features:

User Registration and Login:Provides an option for users to sign up for accounts using Gmail and authenticate themselves.

Parcel Booking:Users can make bookings by specifying information about the sender, receiver, parcel type, weight, and priority.

Parcel Tracking:Users have the ability to track their parcels using a unique tracking number and check the status of booked, in-transit, and delivered parcels.

Previous Shipment Information:Keeps a history of all booked parcels for the convenience of the user.

Notifications:Notifies users of any changes in the status of their parcels as well as any response from the admin.

Profile Management:Enables users to edit their profiles and change passwords.

Support Messaging:Permits users to raise any concerns or inquiries to the admin.

Admin Features :

Admin Dashboard:Displays the overview of the system's activities including stats on parcels and users.

Parcel Management:Admin can view all parcels, update delivery status, and manage shipment details.

User Management:The admin can track and manage registered users.

Analytics & Reports:The dashboard displays performance data like parcel numbers and delivery status.

Support Message Management:The admin is able to read messages from the user and send a reply.

Admin Account Management:The admin creates new admin accounts for managing the system.

Other Features :

JWT Authentication:Secure login and access control mechanism.

Role-Based Access Control:Finds the difference in functions for users and the administrator.

API Integration:Communication between the frontend and backend becomes seamless.

Scalable MERN Stack:Allows room for future modifications and upgrades.

8. ROLES AND RESPONSIBILITIES :

USERS :

Account Responsibility:It is the responsibility of the user to create an account with accurate details and keeping it confidential.

Provision of Correct Information:In case of booking a package, the user needs to give correct details about the sender and receiver for smooth delivery.

Parcel Tracking:It is the responsibility of the user to track the parcel and check its current status via the tracking ID.

Fulfillment of System Requirements:Using a correct email address and communicating through the website system are among other responsibilities of the user.

Communication with Admin:Raising queries or any issue faced by the user can be done through the support message.

Updates on Profile:Updating user profile information helps prevent any issue related to communication and delivering packages.

Admin:

System Maintenance:Keeping the whole system error-free is the responsibility of the admin.

Parcel Status Management:Managing the parcel statuses correctly at different levels is the responsibility of the admin.

Verification of Users:The admin verifies users' accounts to prevent misuse of the system.

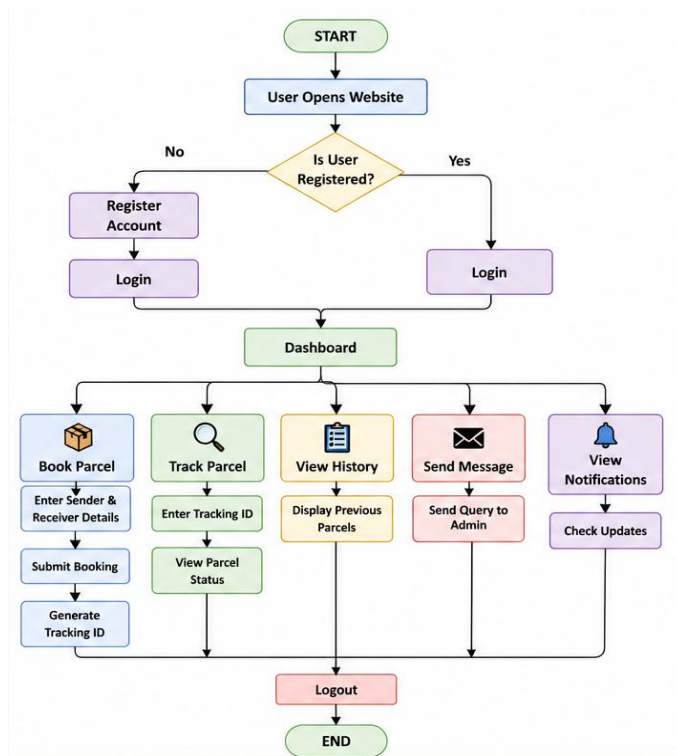
Issues Resolution:The admin needs to solve issues raised by the user and reply promptly.

Data Management:All parcels, users, and messages are managed by the admin inside the system.

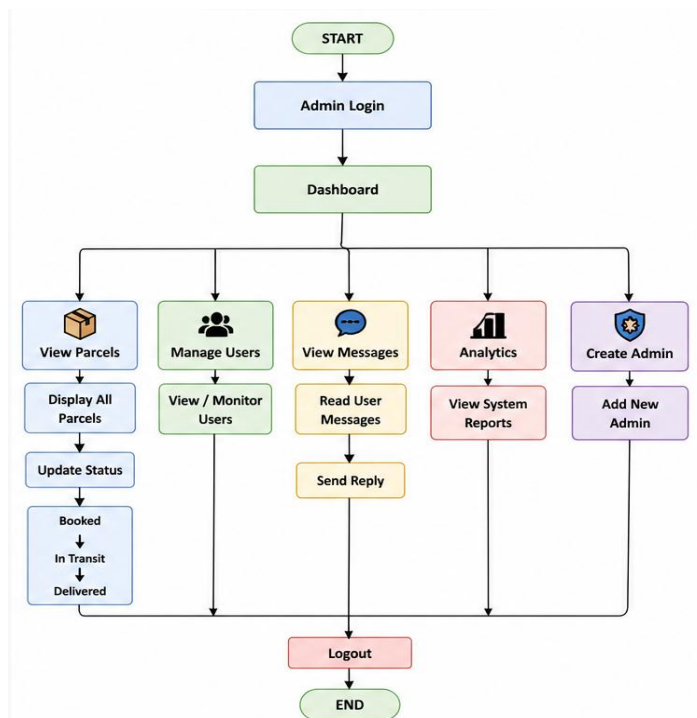
Security of the System:Ensuring security, authenticating users, and data management are among other roles of the admin.

9. FLOWCHART :

USER :

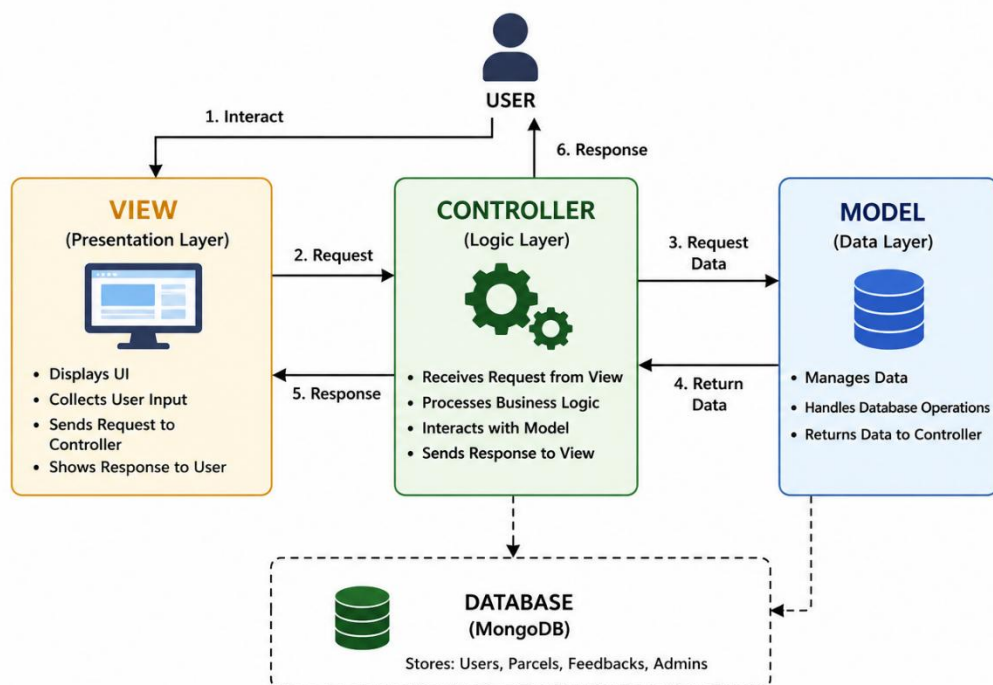


ADMIN :



- User flow begins when the user launches the app. If the user is unregistered, then he registers his account and logs in. The user, upon logging in, gets access to the dashboard. The dashboard contains booking of parcels, tracking of a parcel with a tracking number, viewing past record of parcels, sending messages to admin, and getting notifications. After doing all the necessary steps, the user logs out from the app and thus completes the flow.
- Admin Login Flow starts with Admin Login. Once logged in, the admin is navigated to the Dashboard page where system details are displayed. From there, the admin performs various functions such as checking parcels, tracking their delivery, managing users, messaging them, and reviewing system performance. Finally, once all the work is done, the admin logs out of the system.

10. MVC PATTERN :



Parcelyt Courier and Parcel Management System architecture is based on Model-View-Controller (MVC). This design pattern consists of three main layers: Model, View, and Controller that provide efficient handling of user requests and management of application data.

Model Layer (Data Layer): This layer manages the data and database activities of the application. It works directly with the MongoDB database and handles the storing and retrieving operations of application data such as users, parcels, feedbacks, and admins.

View Layer (Presentation Layer): This layer provides a graphical interface to users. It is built using React.js and helps in showing the data to users and receiving their input via forms like login, booking, and tracking of parcels.

Controller Layer (Logic Layer):The Controller layer sits between the View and the Model layers. It processes user requests according to the business rules and performs communication with the database through the model layer.

Work Process Flow:Firstly, the user initiates actions by interacting with the View, then the request from the view layer is sent to the Controller layer which in turn processes it and sends a request to the database via the Model layer.

Advantages :

Separation of Concerns:It divides the application into three components – Model, View, and Controller, which makes it easier to write clean code and maintain the same.

Ease of Maintenance:Modification in any part of the application will not impact other parts, and it becomes easier for maintenance.

Scalability:It becomes easier to scale the application when there is a need for any additional functionality.

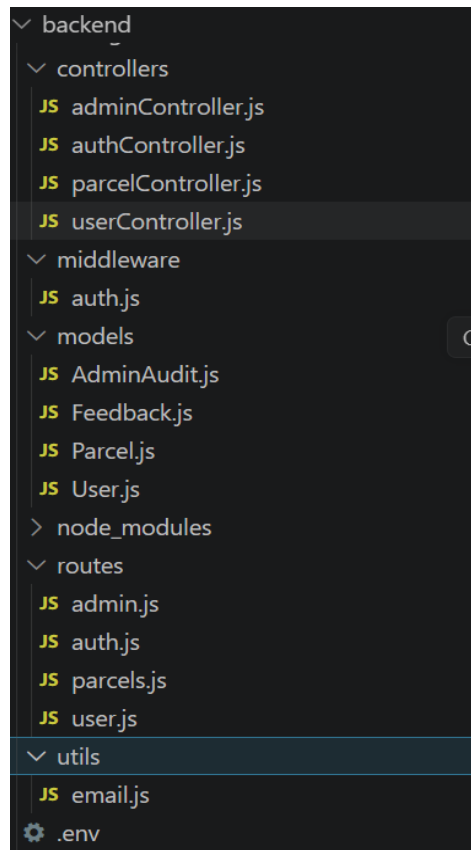
Code Reusability:There is a high possibility that a component can be reused throughout the application.

Efficient Development:More than one developer can develop the different parts of the application such as frontend, backend, and database at the same time.

Testing and Debugging:Every component can be independently tested, which increases the efficiency of testing and debugging.

Flexibility:There is no change needed in the business logic and database when the interface is changed.

11. BACKEND DEVELOPMENT :



Parcelyt Courier and Parcel Management System backend is programmed with Node.js and Express.js. This programming language and framework help in server side logic, API calls, and interaction with the database.

Structure of Backend:

Controllers

- **adminController.js**→Deals with admin operations such as parcel, user management, analytics.
- **authController.js**→Deals with user authentication and log-in through JWT.
- **parcelController.js**→Takes care of the parcel booking, tracking, and shipment history.
- **userController.js** → Deals with user profile, notification, and user operations.

Middleware

- **auth.js** → Deals with authentication and authorization by JWT.

Models

- **User.js**→Defines the schema of the user account (Name, email address, password, etc.).

- **Parcel.js** → Defines the schema of parcel booking information.
- **Feedback.js** → Keeps track of all user messages and admin reply messages.
- **AdminAudit.js** → Keeps the record of admin activity.

Routes

- **auth.js** → Authentication routes for logging in, registering, and other authentications.
- **user.js** → User routes that handle all the activities related to the user such as profiles and notifications.
- **parcels.js** → Parcel routes that manage booking, tracking, and history of parcels.
- **admin.js** → Admin routes used to control all the operations performed by an admin such as controlling parcels and users.

Utilities

- **email.js** → Provides services related to emails such as alerting and notifying.

Environment Configuration

- **.env file** → Used to store environment variables such as URL of the database, server port, etc.

How Backend Works:

- A request is sent to the backend by the frontend through the API routes.
- The relevant route receives this request and forwards it to the controller.
- The controller process the request and works on the database with the help of models.
- Middleware takes care of the authentication and authorization of the request.
- The processed request is then forwarded to the frontend.

Conclusion:

The backend architecture of the project is very well organized and structured.

12. DATABASE DEVELOPMENT :

Parcelyt Courier and Parcel Management System utilizes MongoDB, a NoSQL database that supports storing data in a flexible JSON-like schema. MongoDB is used in managing users' information, parcels' data, tracking records, and support messages.

1. Database Setup:

- MongoDB is accessed using Mongoose, a Node.js database modeling library.
- The database is configured using a connection string in the .env file.

```
const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`DB Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

2. COLLECTIONS (SCHEMAS/MODELS) :

The database consists of the following main collections:

User Collection (User.js):

- Contains information about user accounts
- Fields: name, email, password, phone, address
- Authentication and profile management purposes

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true,
    set: normalizeEmail,
    validate: {
      validator: isGmailAddress,
      message: 'Please use a valid Gmail address'
    }
  },
});
```

Parcel Collection (Parcel.js):

- Contains information about parcel bookings and tracking
- Fields: trackingId, sender information, receiver information, type of parcel, weight, status
- Tracking the progress of parcels

```
const parcelSchema = new mongoose.Schema({
  trackingId: { type: String, required: true, unique: true },
  senderName: { type: String, required: true },
  senderPhone: { type: String, default: '' },
  senderAddress: { type: String, required: true },
  receiverName: { type: String, required: true },
  receiverPhone: { type: String, default: '' },
  receiverAddress: { type: String, required: true },
  parcelType: {
    type: String,
    enum: ['Document', 'Electronics', 'Clothing', 'Food', 'Fragil',
    default: 'Other'
  },
},
```

Feedback Collection (Feedback.js):

- Contains information regarding communications between users and administration
- Fields: message, response, userId, adminId, status
- Supporting a messaging system between users and administrators

```
const feedbackSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  userName: { type: String, required: true },
  userEmail: { type: String, required: true },
  trackingId: { type: String, default: '' },
  issueType: {
    type: String,
    enum: ['Delivery Issue', 'Tracking Mismatch', 'Damaged Parcel', 'Mi',
    default: 'Other'
  },
},
```


Admin Audit Collection (AdminAudit.js):

- Contains information regarding administrative activities
- Fields: createAdmin, updateParcelStatus
- Monitoring system activities

```
const adminAuditSchema = new mongoose.Schema({
  action: { type: String, required: true },
  actorId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  actorName: { type: String, default: '' },
  targetUserId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  targetName: { type: String, default: '' },
  targetEmail: { type: String, default: '' },
  details: { type: String, default: '' }
}, { timestamps: true });
```

3. Database Actions:

- **Insert:** Add new entries for users, parcels, and messages
- **Retrieve:** Fetch information about parcels, users, and notifications
- **Modify:** Modify parcel status, user information, and responses to messages
- **Remove:** Delete any excess data (if necessary)

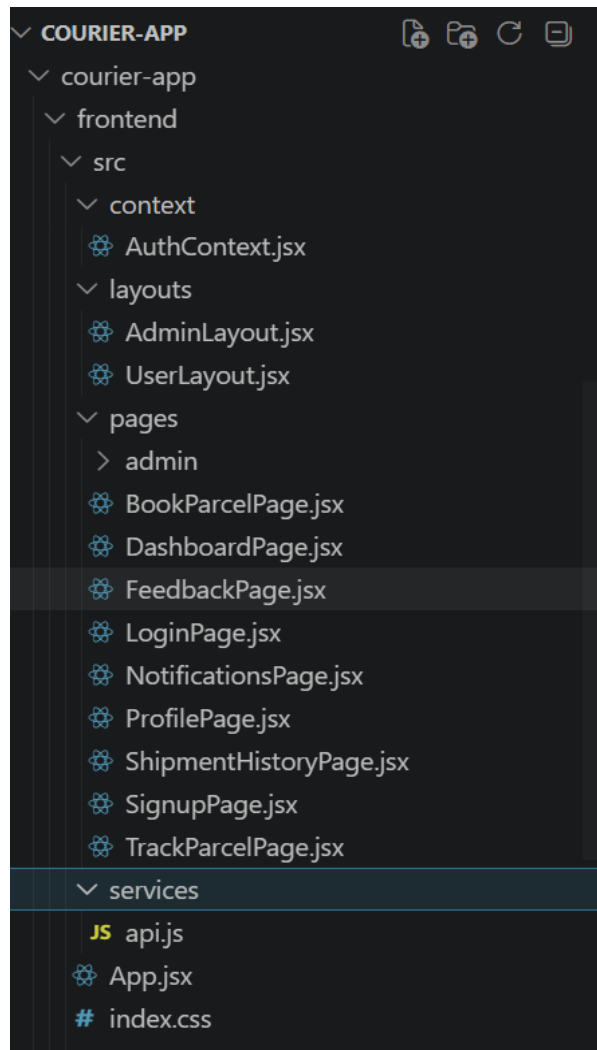
4. Relationships Between the Data Entities:

- The user table is related to the parcel table with userId
- The feedback table is associated with both userId and adminId
- Parcel tracking is facilitated by trackingId

5. Data Processing:

- Data validation is done via Mongoose schema definitions
- A consistent structure exists for each collection
- Optimized queries are utilized for data access

13. FRONTEND DEVELOPMENT :



The Parcelyt Courier and Parcel Management System frontend is created using the React.js framework, allowing for a highly interactive user interface. It is responsible for presenting data to the user and interacting with him/her through APIs for interaction with the backend.

Frontend Structure:

Context

- **AuthContext.jsx** → This manages the authentication state for the entire app. It stores the login state of the user and his session.

Layouts

- **AdminLayout.jsx** → Layout for admin-only pages like dashboard and management panels.
- **UserLayout.jsx** → Layout for pages specific to users that navigate the page and contain certain content.

Pages

- **LoginPage.jsx** → Page for user and admin login process.
- **SignupPage.jsx** → Page where users can register themselves in the system.
- **DashboardPage.jsx** → Dashboard showing all information about the user's parcels.
- **BookParcelPage.jsx** → Page for entering parcel information and booking a shipment.
- **TrackParcelPage.jsx** → Allows tracking of parcels with the help of their ID.
- **ShipmentHistoryPage.jsx** → History of parcels booked by the user.
- **ProfilePage.jsx** → Page to edit personal details of a user.
- **NotificationsPage.jsx** → Page for showing system notifications.
- **FeedbackPage.jsx** → User communication with admin through messages.
- **admin/** → All admin-specific pages like parcel management and analytics.

Services

- **api.js** → For making API calls between frontend and backend.

Main Files

- **App.jsx** → The main file containing the application components and routing.
- **index.css** → The CSS for the application.

Flow of Frontend:

- Users interact with the interface via various pages.
- User inputs such as logins, bookings, and tracking are received by React components.
- An API call is made via the service layer (api.js).
- Processing takes place at the backend, and a response is received.
- Based on the response received from the backend, the frontend updates the UI.

Frontend Features:

- Responsive and intuitive user interface
- Separate layouts for users and admins
- Live update capabilities via APIs
- Context-based security
- Component-based design

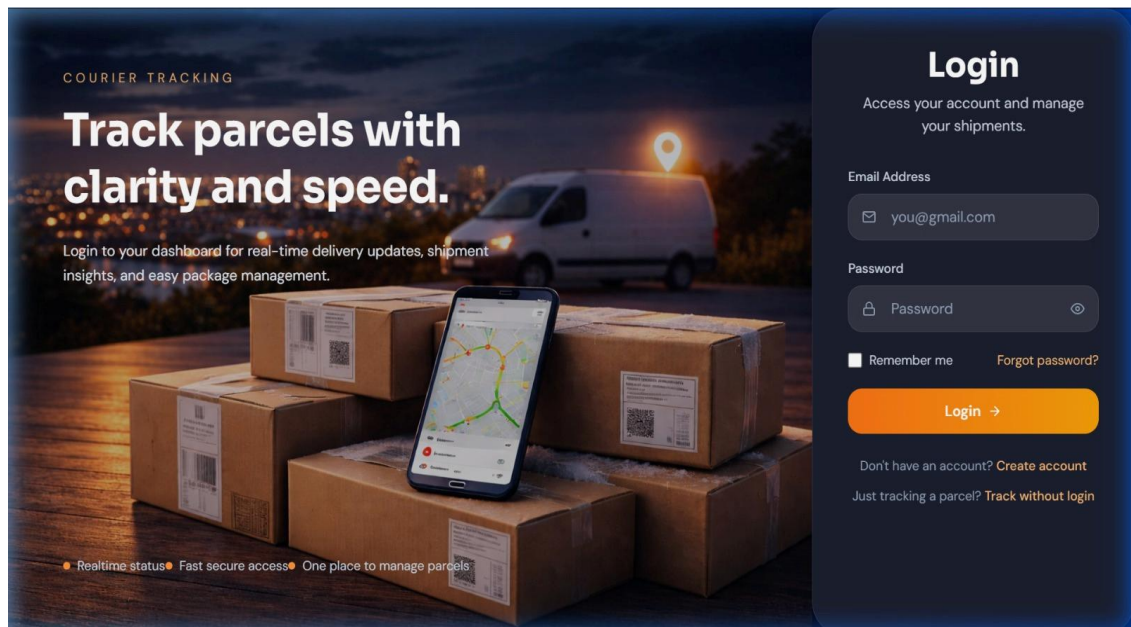
Conclusion:

The frontend has been implemented using React.js, which includes page, layout, and services separation.

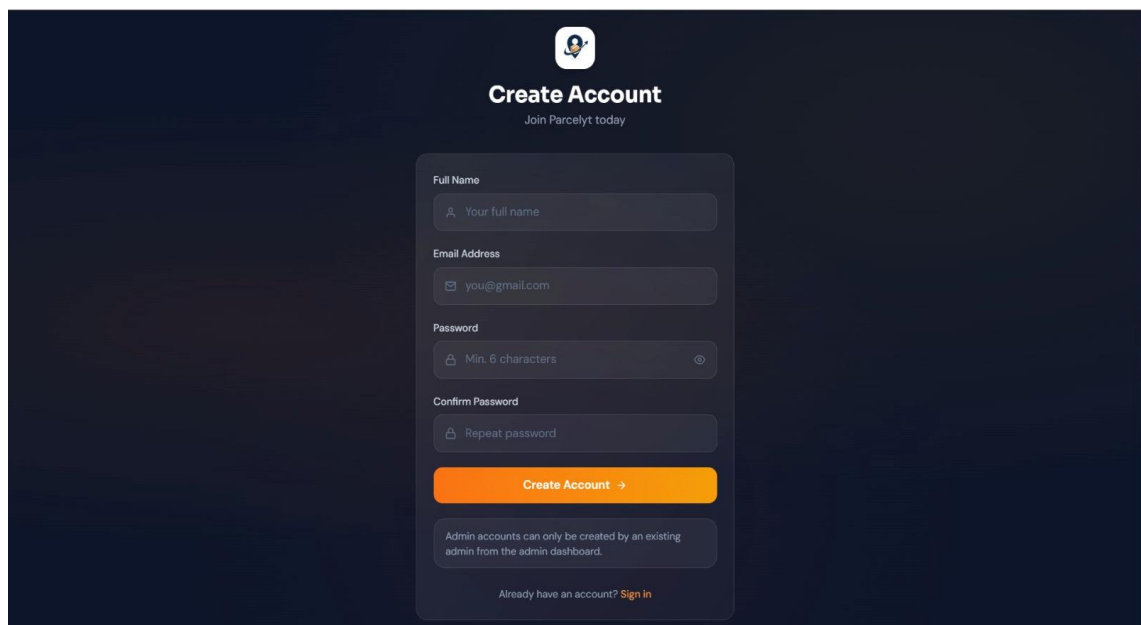
14. OUTPUT SCREENSHOTS :

1. USER SIDE :

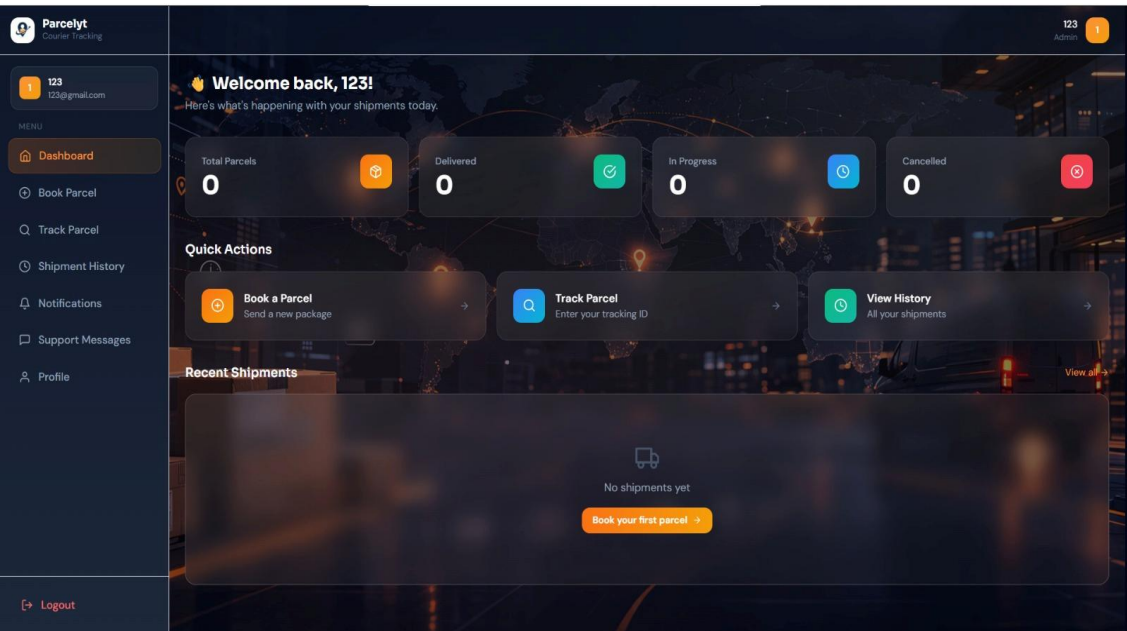
LOGIN PAGE :



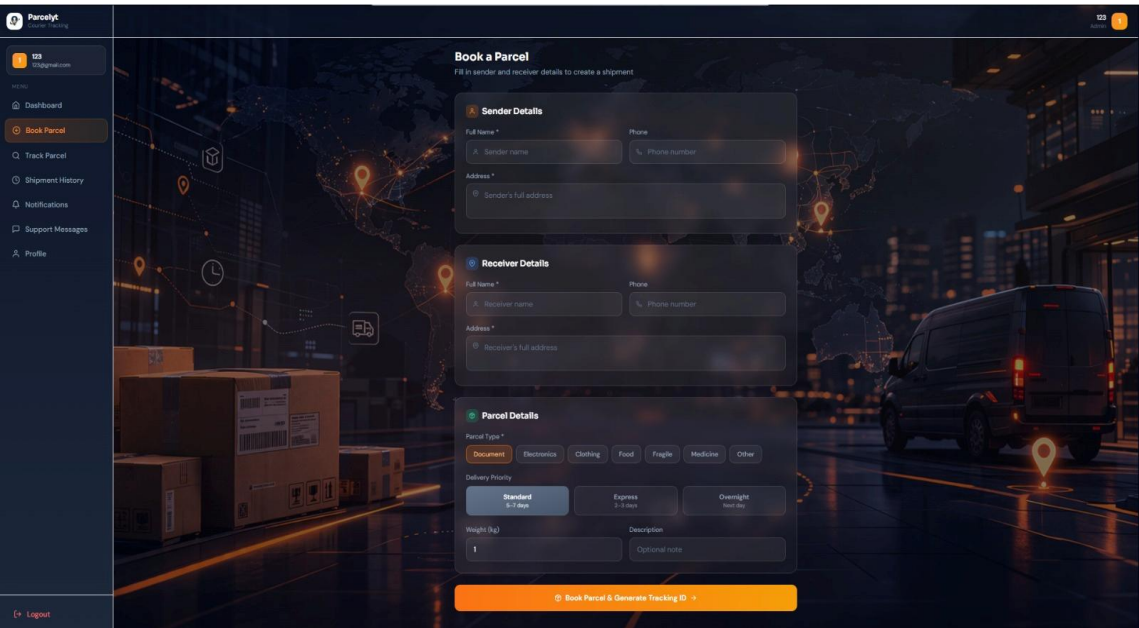
SIGNUP PAGE :



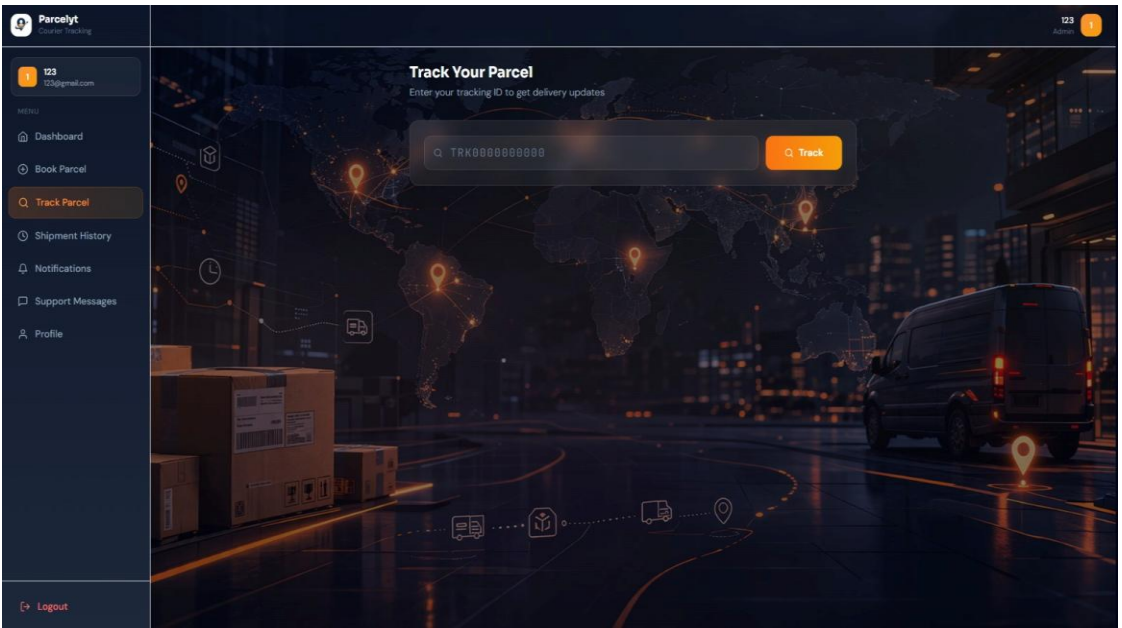
USER DASHBOARD PAGE :



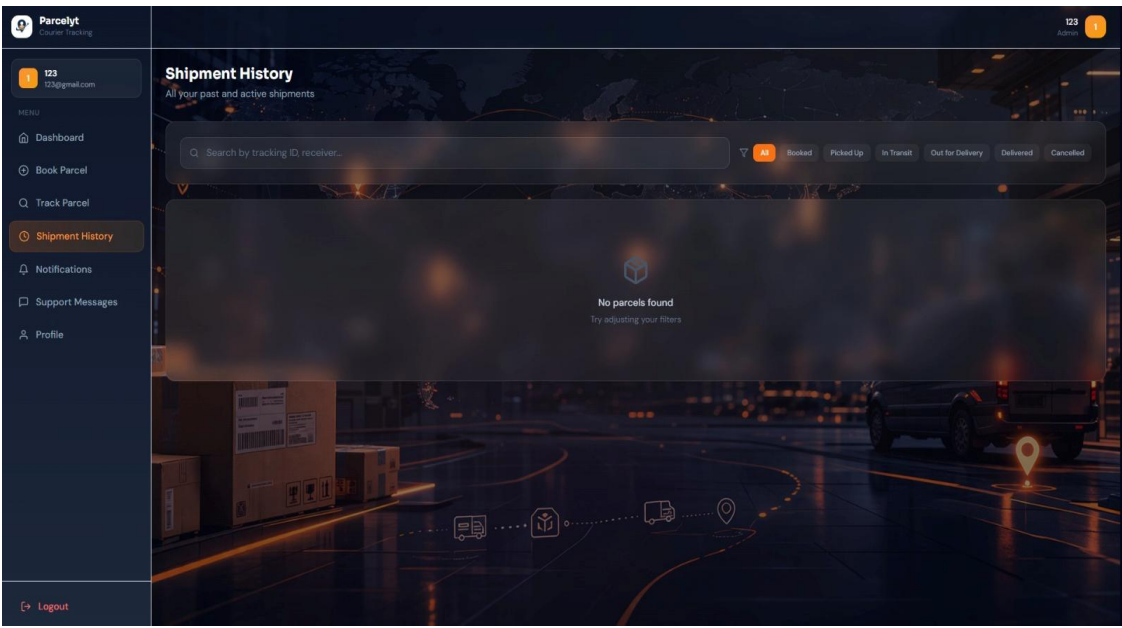
BOOK PARCEL PAGE :



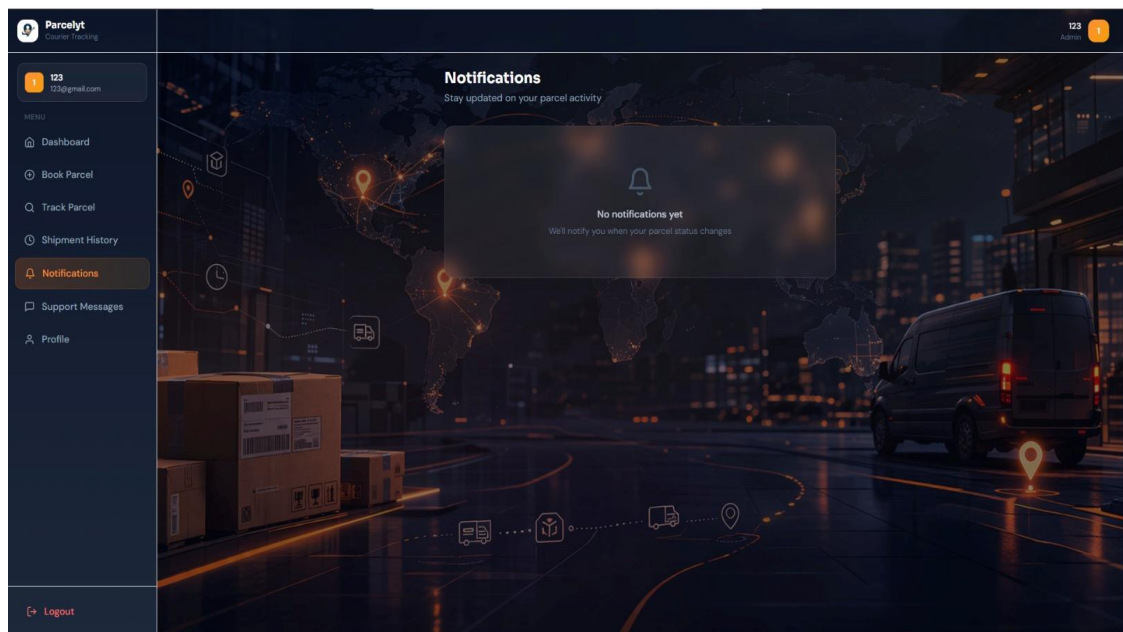
TRACK PARCEL PAGE :



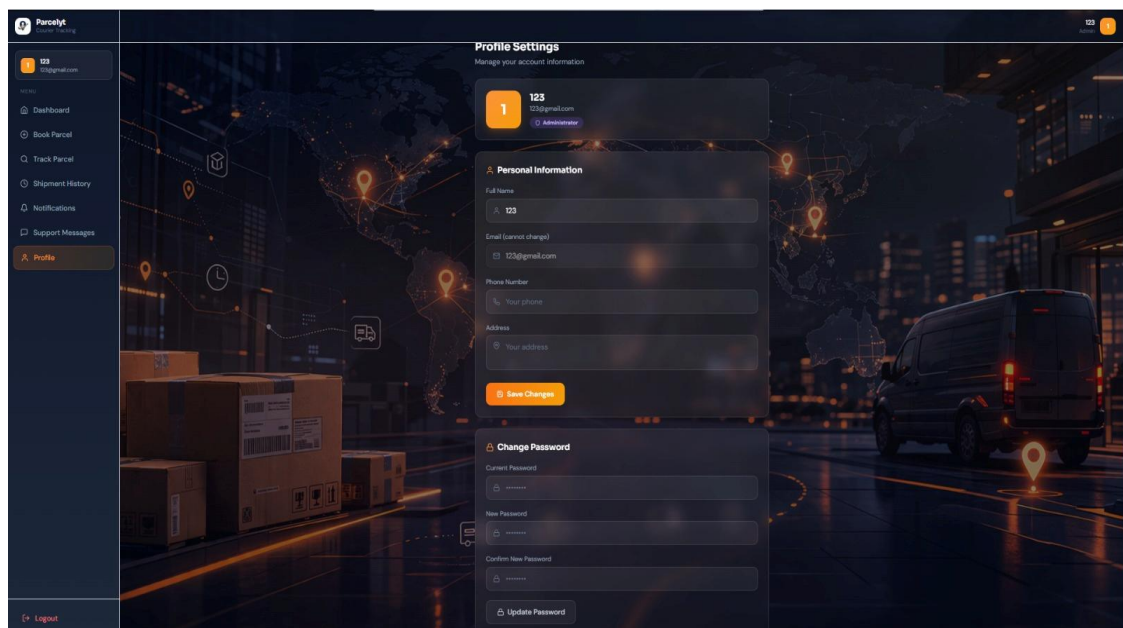
SHIPMENT HISTORY PAGE :



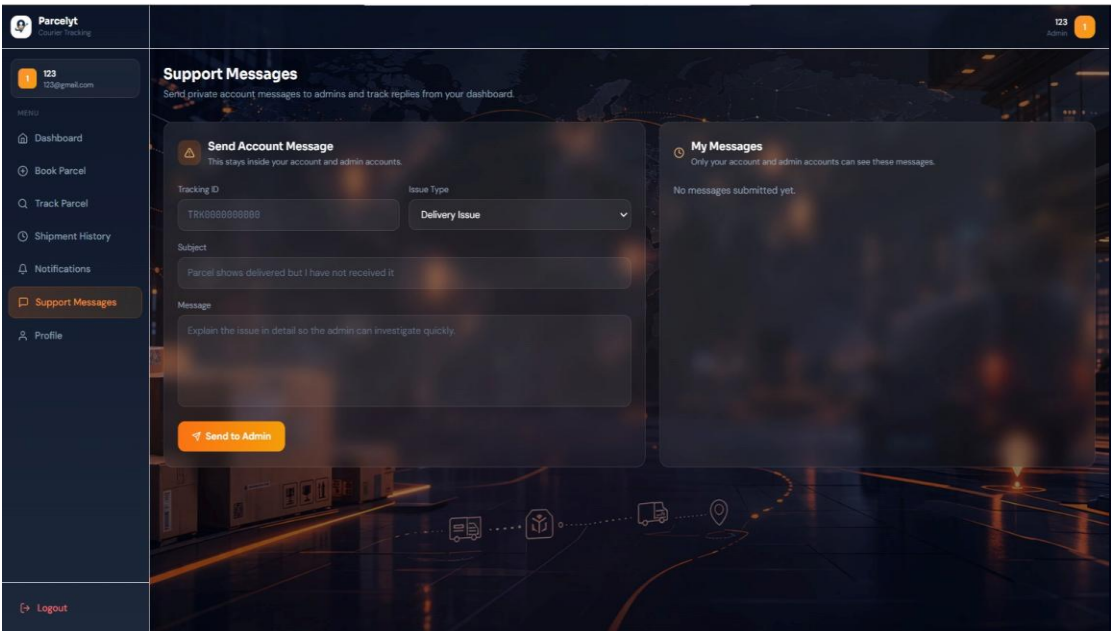
NOTIFICATIONS PAGE :



PROFILE PAGE :

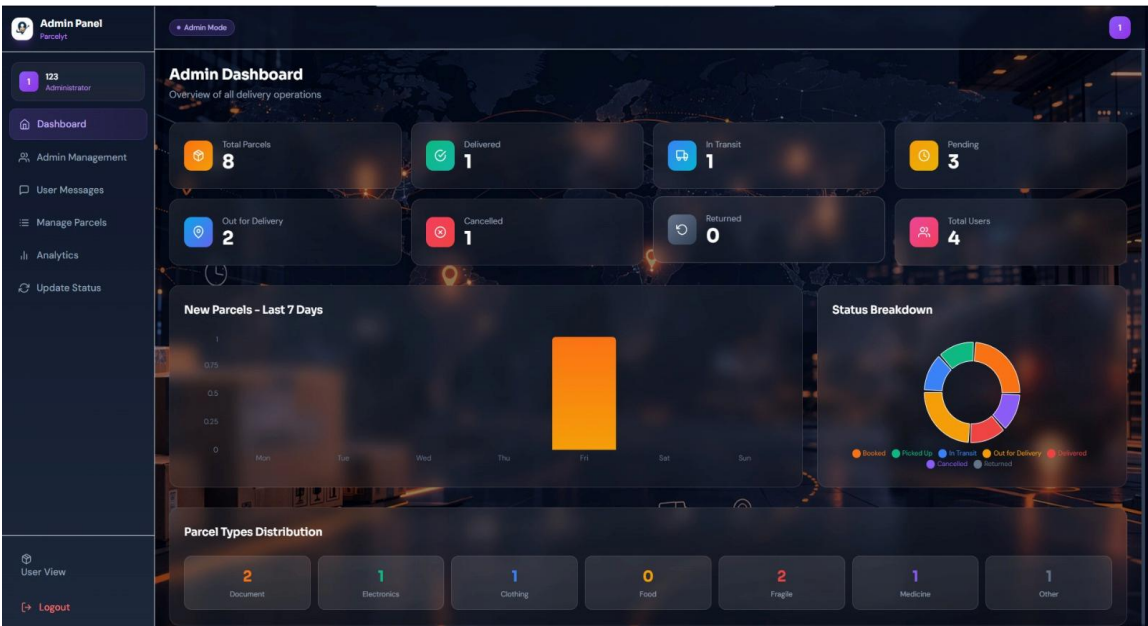


FEEDBACK/MESSAGE PAGE :

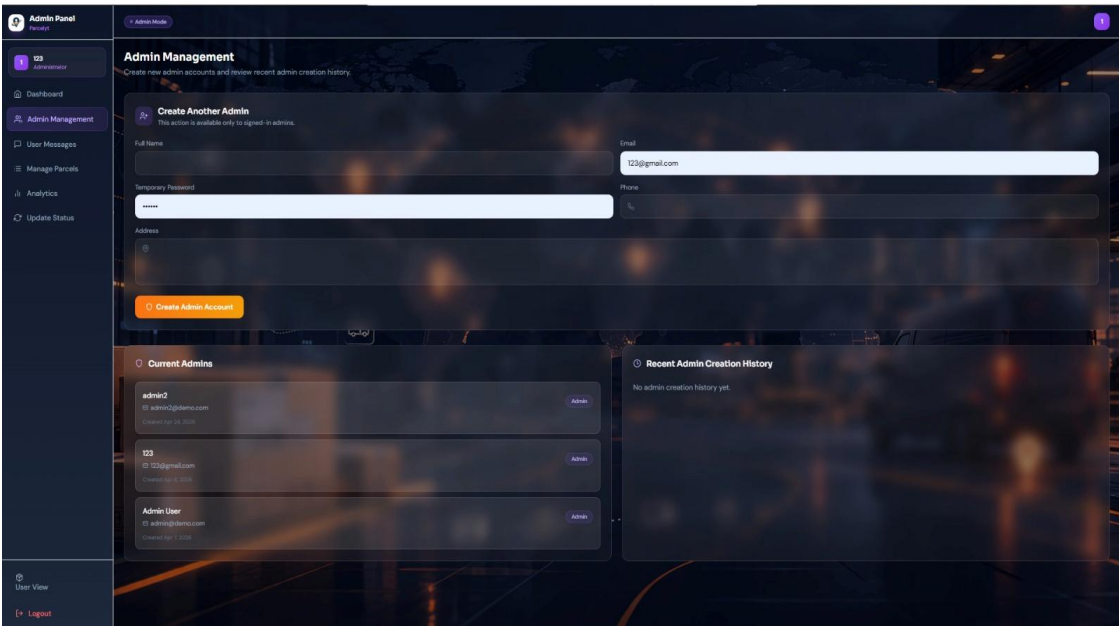


2. ADMIN SIDE :

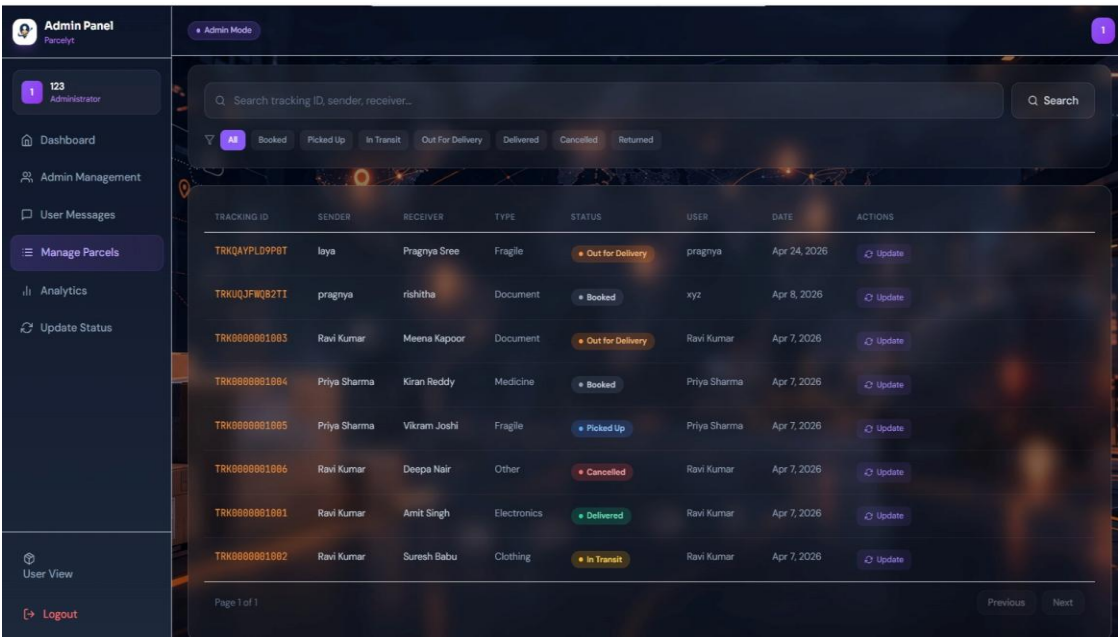
ADMIN DASHBOARD PAGE :



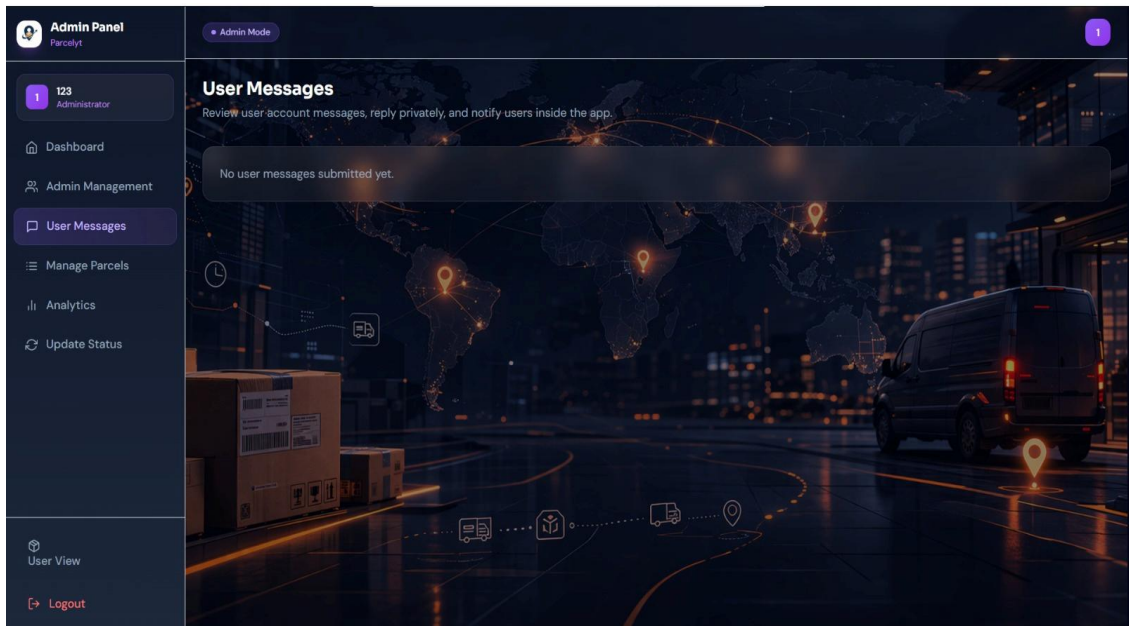
ADMIN MANAGEMENT PAGE :



PARCEL MANAGEMENT PAGE :



ADMIN MESSAGES PAGE :



14. CONCLUSION :

Parcelyt Courier and Parcel Management System is an efficient solution to manage all processes related to courier services in a digital way. The application makes it easier to book, track, and monitor deliveries by offering a convenient interface and relevant information. It saves time and efforts, increasing the level of accuracy and transparency between users and administrators.

Parcelyt is created using a powerful MERN framework, making it easy to scale, flexible to adapt, and high performing. Such features as authentication, parcel tracking via unique IDs, notifications, and messaging make using the app more pleasant.

MVC architecture is chosen to ensure good organization and structure of a project, while backend is created following the same rules. Properly integrated frontend and backend guarantee effective communication and exchange.

In conclusion, we have created a modern website that solves a real-life problem and can be used for further development. There are multiple ideas that can be implemented to improve the functionality of our project: online payments, GPS tracking, and a mobile application.

15. GITHUB REPOSITORY :

https://github.com/kundanavadlamudi/courier_and_parcel_tracking_system

16. VIDEO LINK :

<https://drive.google.com/file/d/1-FGD3pe0WcuV0OaeD-ubweXU-6SSN0p7/view?usp=sharing>

https://drive.google.com/file/d/1JsHCVHIox4hc91_j3p9zOwIApsvLl4xT/view?usp=drive_link

https://drive.google.com/file/d/1XrSkasIwpDnGtXJNNDkWmzMUmUPV7NDd/view?usp=drive_link